

The rsedit Lisp Interpreter

Machinery, semantics, and how to embed it

An embeddable Lisp that knows nothing about text editing. This documents the reader, the evaluator and the host contract - the three things that make up the interpreter itself. The standard library and everything the editor adds are out of scope: they are users of the interpreter, not part of it.

Contents

1 What this is

2 Values

- 2.1 Form and Cons: the distinction that matters most
- 2.2 nil, truth, and the empty list
- 2.3 Printing

3 The reader

- 3.1 Lexing
- 3.2 Literal syntax
- 3.3 The three rewrites

4 Evaluation

- 4.1 What eval_step does, in order
- 4.2 Dispatching a Form
- 4.3 Which forms preserve tail position

5 Special forms

- 5.1 Quoting
- 5.2 Conditionals
- 5.3 Sequencing
- 5.4 Non-local exit
- 5.5 Binding
- 5.6 Iteration
- 5.7 Definition
- 5.8 Concurrency

6 Environments and scope

- 6.1 Scope is lexical
- 6.2 The one reference cycle

7 Calling

- 7.1 Macros: expand now, evaluate the expansion after
- 7.2 Functions: evaluate the arguments, then enter the body

8 Errors and non-local exit

- 8.1 The error type
- 8.2 Two mechanisms, deliberately different
- 8.3 catch and throw
- 8.4 condition-case
- 8.5 Raising from Lisp
- 8.6 What survives an unwind

9 The host contract

10 Metering

11 Concurrency

11.1 spawn

11.2 Fibers

11.3 Atoms

12 Embedding it

13 Where the interpreter stands

13.1 In the tree

13.2 Written and verified, not yet applied

13.3 Agreed next, not yet written

1 What this is

`core/src/lisp/` is a small Lisp interpreter that knows nothing about text editing. It is generic over a host type `T`, and everything it needs from that host is declared in one trait, `LispContext`. The editor is just one host; the test suite uses Rust's unit type `()` as another.

There are three layers, and they are worth keeping separate in your head:

LAYER	WHERE	JOB
Reader	Parser in <code>lisp.rs</code>	text to a syntax tree, in one pass
Evaluator	<code>eval / eval_step</code> in <code>lisp.rs</code>	syntax tree to a value, on a trampoline
Host contract	trait <code>LispContext</code>	metering, diagnostics, call frames

This document covers those three and nothing else. The standard library installed by `setup_base_env` (`car`, `+`, `format`, and roughly seventy others) and everything the editor adds on top are deliberately out of scope: they are *users* of the interpreter, not part of it. The one place they are mentioned is the embedding example in section 12, where you need a library to have anything to call.

Scope of accuracy Everything here describes commit `d4a5ac2` as it stands. Section 13 lists the work that has been written and verified but is not yet applied to the tree, so that the difference between the two is explicit rather than something you have to reconstruct.

2 Values

Every value in the language is one `LispExp<T>`. The type parameter is the host context; it appears because lambdas capture environments and environments hold values, so the whole value tree is parameterised by the host it belongs to.

VARIANT	PAYLOAD	WHAT IT IS
Form	<code>Arc<Vec<LispExp<T>>></code>	a <i>syntax</i> node - what the reader builds and <code>eval</code> dispatches on
Cons	<code>Arc<ConsCell<T>></code>	a <i>data</i> list: one cell, <code>car</code> and <code>cdr</code>
Vector	<code>Arc<Vec<LispExp<T>>></code>	<code>[1 2 3]</code>
Map	<code>Arc<HashMap<String, LispExp<T>>></code>	<code>{key value}</code>
Number	<code>f64</code>	every number, integral or not
Symbol	<code>Arc<String></code>	identifiers, including <code>nil</code> and <code>t</code>
String	<code>Arc<String></code>	string literals
Lambda	<code>Arc<Lambda<T>></code>	a closure: parameters, body, captured environment
Primitive	<code>fn pointer + doc</code>	a native function
Atom	<code>Arc<RwLock<LispExp<T>>></code>	the one mutable cell in the language
Fiber	<code>Arc<RwLock<FiberState<T>>></code>	a resumable suspended body

Every payload is behind an `Arc`, so cloning a `LispExp` is a refcount bump or an `f64` copy. That is what lets values cross into a spawned thread and be shared rather than deep-copied.

2.1 Form and Cons: the distinction that matters most

`Form` and `Cons` are both lists, and confusing them is the single easiest way to write a bug in this interpreter. The rule is short:

- `Form` is *code*. The reader produces it, `eval` dispatches on it, and it is never a runtime value.
- `Cons` is *data*. It is what Lisp code computes with, and `eval` never dispatches on it directly.

They are not interchangeable, and exactly two functions convert between them:

FUNCTION	DIRECTION	WHEN IT RUNS
<code>form_to_data</code>	Form to Cons	at <i>read</i> time, over quoted structure only
<code>data_to_form</code>	Cons to Form	when data reaches evaluation position

The reader converting quoted structure once, at read time, is what makes `'(1 2 3)` free in a loop: the conversion already happened, and evaluating the quote is a refcount bump. `data_to_form` is the rarer

direction, reached by `(eval some-list)` and by macro expansions, which produce data that then has to be evaluated as code.

The one exception inside ``data_to_form`` `(quote X)` is not descended into. `X` is data and must stay data - both when the reader builds the form and when a macro builds one by hand with `(cons 'quote (cons items nil))`. Without that carve-out, `quote` would hand a syntax node back as a value.

2.2 nil, truth, and the empty list

`nil` is the symbol `nil`. It is the empty list, the false value, and self-evaluating. So is `t`, and so is any symbol starting with `:`. There is no separate boolean type and no separate empty-list representation - `(list)`, `'()` and `nil` are all the same value.

```
(list)      => nil
'()         => nil
(cons 1 2)   => (1 . 2)
(cons 1 (cons 2 3)) => (1 2 . 3)
```

2.3 Printing

`LispExp` implements `Debug` by hand rather than deriving it, because a derived `Debug` renders a cons chain as one nested `Cons(ConsCell { car: .., cdr: Cons(..) })` per element - unreadable in an echo area, which is where most of these strings end up. The hand-written one prints Lisp source:

VALUE	PRINTS AS
a list, proper or dotted	<code>(1 2 3)</code> , <code>(1 2 . 3)</code>
a vector	<code>[1 2]</code>
a map	<code>{key value}</code>
an integral number	3, not 3.0 - but 1.5 stays 1.5
a lambda	<code>#<lambda (a &optional b &rest c)></code>
an atom	<code>#<atom></code> - deliberately opaque, see below
a fiber	<code>#<fiber></code> / <code>#<fiber done></code> / <code>#<fiber running></code>

Why atoms print opaquely An atom is the only container that can be made to hold itself. Recursing into it could hang the printer, and taking its lock could deadlock against a writer that is formatting its own contents. `deref` is how you look inside one.

3 The reader

`Parser::new(source)` wraps a `Peekable<Chars>`; `parser.next()` returns one `LispExp` per top-level form. Lexing and parsing are interleaved - there is no separate token vector.

3.1 Lexing

`next_token` is a nine-state machine over the character stream. The states name what the lexer is in the middle of: `Default`, `InComment`, `InSymbol`, `InString`, `InStringSlash`, `InNumber`, `InNumberMinusStart`, `InNumberAfterDot`, `InDotStart`. The last three exist because `-`, `.` and a digit are each ambiguous until the next character arrives: `-` may start a negative number or be the symbol `-`, and `.` may start a float or be the dotted-pair marker.

Sixteen token kinds come out, including a `parens_stack` that checks balance as it goes, so `(a ]` fails at the bracket rather than at end of input.

3.2 Literal syntax

WRITTEN	READS AS	NOTE
<code>(a b c)</code>	<code>Form</code>	code
<code>[1 2 3]</code>	<code>Vector</code>	elements are evaluated when the vector is
<code>{k 1 j 2}</code>	<code>Map</code>	keys are symbols or strings, values any form
<code>(a . b)</code>	improper Cons chain	the dotted pair
<code>"text"</code>	<code>String</code>	<code>\\</code> escapes handled in <code>InStringSlash</code>
<code>;; comment</code>	nothing	to end of line
<code>'x</code>	<code>(quote x)</code>	with <code>x</code> converted to data at read time
a backtick before <code>x</code>	<code>(backquote x)</code>	template - see the code block below
<code>,x</code>	<code>(unquote x)</code>	only meaningful inside a backquote
<code>,@x</code>	<code>(unquote-splicing x)</code>	splices a list's elements

3.3 The three rewrites

The reader is not purely structural: it rewrites three shapes as it goes. These are the only places the reader makes a semantic decision.

```
'X      -> (quote form_to_data(X)) ; converted to data here, once
`X      -> (backquote X)           ; left as syntax; the template is walked at eval t:
(a . b) -> improper_list([a], b)   ; a cons chain, not a special variant
```

4 Evaluation

`eval` is an explicit trampoline. It never recurses to continue a computation whose result is its own result:

```
pub fn eval<T: LispContext>(exp: &LispExp<T>, env: Arc<Env<T>>, ctx: &T)
    -> Result<LispExp<T>, EvalError>
{
    let mut current_exp = exp.clone();
    let mut current_env = env;
    loop {
        match eval_step(&current_exp, current_env.clone(), ctx)? {
            EvalStep::Done(result) => return Ok(result),
            EvalStep::TailCall(next_exp, next_env) => {
                current_exp = next_exp;
                current_env = next_env;
            }
        }
    }
}
```

`eval_step` performs exactly one reduction and reports which kind it was. `Done` means the value is final. `TailCall` means "my result is whatever this other expression evaluates to", and the loop rebinds its two locals rather than growing the Rust stack. A tail-recursive Lisp function therefore runs in one Rust frame however deep it goes, and the limit on recursion is the fuel budget rather than the native stack.

4.1 What `eval_step` does, in order

The first statement is the fuel charge, before the match, so every reduction is metered whichever branch runs - loops, calls and argument evaluation alike:

```
fn eval_step<T: LispContext>(exp: &LispExp<T>, env: Arc<Env<T>>, ctx: &T)
    -> Result<EvalStep<T>, EvalError>
{
    ctx.consume_fuel(1)?;
    match exp { ... }
}
```


EXP IS	RESULT
Number String Lambda Primitive Atom Fiber	Done(exp.clone()) - self-evaluating
Symbol that is nil, t, or starts with :	Done(exp.clone()) - self-evaluating too
any other Symbol	env.get_variable(name), else Err(UnboundVariable)
Cons	TailCall(data_to_form(exp)?, env) - data in evaluation position
Vector	every element evaluated, Done(Vector)
Map	every value evaluated, Done(Map)
empty Form	Done(nil) - () is the empty list
non-empty Form	dispatch on the head, below

4.2 Dispatching a Form

items[0] decides the shape of the call. There are exactly four cases:

HEAD	WHAT HAPPENS
Symbol(s)	eval_special_form_or_call_step - 23 special forms by name, then the macro namespace, then the function namespace
Form(..)	the head is evaluated first, spliced back in front of the arguments, and the rebuilt form is tail-called - this is how ((f) 1 2) works
Lambda(l)	a literal lambda in head position: arguments are evaluated and it is called directly
anything else	Err(InvalidFunctionCall)

4.3 Which forms preserve tail position

A form preserves tail position when its last subexpression is handed back as TailCall rather than evaluated inline. All the ones you would expect do:

PRESERVES TAIL POSITION	DOES NOT, AND MUST NOT
if progn let let* cond and or when unless	unwind-protect - it has to stay on the Rust stack to run its cleanups

while, dolist and dotimes iterate internally and do not need it. quote, setq, defun, lambda, defmacro, defvar, spawn and fiber all produce a value directly.

```
;; Runs in constant Rust stack: the recursive call is in tail position
;; inside `if`, which hands it back to the trampoline.
(defun count-down (n)
  (if (= n 0) 'done (count-down (- n 1))))
(count-down 100000) => done
```

5 Special forms

Twenty-five names are matched before anything else. Their arguments are *not* evaluated first - each arm decides what to evaluate and when, which is exactly what makes them special forms rather than functions. A name that matches none of them falls through to the macro namespace and then the function namespace.

5.1 Quoting

FORM	MEANING
(quote X)	returns X unevaluated. The reader already converted X to data.
(backquote X)	template: (unquote E) inserts E's value, (unquote-splicing E) splices E's elements. One level of nesting.

```
'(1 2 3)                => (1 2 3)
`(1 ,(+ 1 1) 3)          => (1 2 3)
`(1 ,@(list 2 3) 4)      => (1 2 3 4)
`(a (b ,(+ 1 2)) [c ,(+ 3 4)]) => (a (b 3) [c 7])
```

5.2 Conditionals

FORM	MEANING
(if COND THEN [ELSE...])	ELSE forms run as an implicit progn
(cond (TEST BODY...)...)	first true clause wins; a clause with no body yields its test's value
(when COND BODY...)	BODY as progn when COND is true, else nil
(unless COND BODY...)	the inverse
(and FORM...)	left to right, stops at the first nil, yields the last value
(or FORM...)	left to right, stops at the first non-nil

```
(cond ((= 1 2) 'no)
      ((= 1 1) 'yes)
      (t      'fallback)) => yes
(and 1 2 3)                => 3
(or nil nil 7)             => 7
```

5.3 Sequencing

FORM	MEANING
(progn BODY...)	evaluates each, yields the last
(unwind-protect BODY CLEANUP...)	CLEANUP runs whether BODY succeeded or failed; BODY's result or error is what propagates

```
(setq trace nil)
(unwind-protect (undefined-function)    ; fails
  (setq trace 'cleaned))                ; still runs
;; the error propagates, and `trace` is 'cleaned
```

5.4 Non-local exit

FORM	MEANING
(catch TAG BODY...)	evaluates TAG, then BODY; returns the last body value, or the value of a throw whose tag matches
(condition-case VAR BODY HANDLER...)	evaluates BODY; on failure runs the first HANDLER whose condition matches, with VAR bound to the error

Their partners `throw` and `signal` are *functions*, not special forms - both arguments are evaluated, so they compose with `funcall` and `apply` the way they do in Emacs. Section 8 covers the pair of mechanisms in full, including the rule about which one intercepts what.

```
(catch 'done
  (dolist (x '(1 2 3 4))
    (if (= x 3) (throw 'done x))))    => 3

(condition-case e (car 5)
  (wrong-type-argument (nth 1 (cdr e)))) => 5
```

Neither preserves tail position Both evaluate their protected body with `eval` directly rather than handing it back as a `TailCall`, because the trampoline would run it outside their Rust frame and the failure would escape. Real Emacs has the same property.

5.5 Binding

FORM	MEANING
(let ((VAR VAL)...) BODY...)	binds in a new child environment; all VALs evaluated in the <i>outer</i> one
(let* ((VAR VAL)...) BODY...)	each VAL sees the bindings before it
(setq SYM VAL ...)	assigns in the nearest environment that already has SYM, else the current one
(defvar SYM [VAL])	assigns only if SYM is unbound
(defconst SYM VAL)	assigns unconditionally

A bare symbol is a valid binding and means nil, so (let (x) x) yields nil.

```
(let ((a 1) (b 2)) (+ a b)) => 3
(let* ((a 1) (b (+ a 1))) b) => 2
(let (x) x) => nil
```

5.6 Iteration

FORM	MEANING
(while COND BODY...)	yields nil
(dolist (VAR LIST [RESULT]) BODY...)	VAR is bound in a fresh child env; RESULT is evaluated after BODY...
(dotimes (VAR COUNT [RESULT]) BODY...)	VAR counts 0 up to but excluding COUNT

```
(setq acc nil)
(dolist (x '(1 2 3) acc)
  (setq acc (cons x acc))) => (3 2 1)
```

5.7 Definition

FORM	MEANING
(defun NAME PARAMS BODY...)	defines in the <i>function</i> namespace
(lambda PARAMS BODY...)	yields a closure capturing the current environment
(defmacro NAME PARAMS BODY...)	defines in the <i>macro</i> namespace

The parameter grammar is the same for all three: required names, then optionally `&optional` names, then optionally `&rest` and exactly one name. Markers out of order, or `&rest` with zero or several names, are errors at definition time rather than at call time.

```
(defun greet (name &optional greeting &rest extras)
  (list name greeting extras))
```

```
(greet "a")           => ("a" nil nil)
(greet "a" "hi")      => ("a" "hi" nil)
(greet "a" "hi" 1 2)  => ("a" "hi" (1 2))
```

5.8 Concurrency

FORM	MEANING
(spawn LAMBDA)	runs LAMBDA's body on a new OS thread, returns nil immediately, never joins
(fiber BODY...)	captures BODY and a fresh child environment as a suspended value

Both are covered in section 11.

6 Environments and scope

```
pub struct Env<T: LispContext> {
    pub variables: RwLock<HashMap<String, LispExp<T>>>,
    pub functions: RwLock<HashMap<String, LispExp<T>>>,
    pub macros:    RwLock<HashMap<String, LispExp<T>>>,
    pub parent:    Option<Arc<Env<T>>>,
}
```

Three separate maps make this a Lisp-2: `(f x)` and `x` can name completely different things, and `get_variable`, `get_function` and `get_macro` are three independent lookups. Each walks the parent chain to the root and returns a clone.

6.1 Scope is lexical

A `Lambda` stores `env: Arc<Env<T>>` - the environment in force where it was *defined*. Calling it evaluates the body in `Env::new_child(&lambda.env)`, never in the caller's environment. That is lexical scoping, and it is the same rule modern Emacs uses under `lexical-binding: t`.

```
(setq x 'outer)
(defun show () x)           ; captures the environment where it was defined
(let ((x 'inner)) (show)) => outer ; the let binding is invisible to `show`
```

What follows from that There are no special variables: a `let` cannot establish ambient context for a callee. The equivalent is an explicit parameter, or a save-and-restore around a global with `unwind-protect`.

6.2 The one reference cycle

`defun` stores the `Lambda` in the same `Env` the `Lambda` holds an `Arc` to, so every top-level function closes a reference cycle that `Arc` cannot collect. For the global environment this is harmless - it outlives the process anyway. It is a genuine leak only for a closure stored back into a `let` frame, or an `Atom` made to contain itself.

7 Calling

A Form whose head is a symbol is tried against three things in order: the 23 special forms, then the macro namespace, then the function namespace. Each layer has a different rule about when arguments are evaluated, and that is the whole of the difference between them.

7.1 Macros: expand now, evaluate the expansion after

```
let frame = Env::new_child(&macro_lambda.env);
bind_lambda_args(&macro_lambda, args, &frame)?; // args RAW, unevaluated

let mut expansion = nil;
for form in &macro_lambda.body {
    expansion = eval(form, frame.clone(), ctx)?;
}
TailCall(expansion, env) // the CALLER's environment
```

Three things to notice. The arguments arrive unevaluated, as the raw call-site syntax. The expansion is evaluated in the *caller's* environment, not the macro's. And no call frame is pushed, so macros never appear in a backtrace.

No expansion cache The body is re-expanded on every call. For a macro in a hot loop that is real work repeated for no reason - the obvious place to start if macro-heavy code ever shows up in a profile.

```
(defmacro my-unless (cond &rest body)
  `(if ,cond nil (progn ,@body)))

(my-unless nil 1 2 3) => 3
```

7.2 Functions: evaluate the arguments, then enter the body

```
let args = args.map(|a| eval(a, env.clone(), ctx)?); // strict, left to right
ctx.push_call_frame(symbol);

Lambda    let frame = Env::new_child(&l.env);          // lexical: l.env, not env
          bind_lambda_args(&l, &args, &frame)?;
          for f in &l.body[..n-1] { eval(f, frame.clone(), ctx)?; }
          ctx.pop_call_frame();                        // BEFORE the tail call
          TailCall(l.body.last(), frame)

Primitive let r = pointer(&args, env.clone(), ctx)?;
          ctx.pop_call_frame(); Done(r)
```

A primitive is a plain function pointer with the signature `fn(&[LispExp<T>], Arc<Env<T>>, &T) -> Result<LispExp<T>, EvalError>`. It receives its arguments already evaluated, as a slice - which is why changing how lists are represented never changes a primitive's signature.

THREE CONSEQUENCES OF THAT EXACT ORDERING

- The frame is popped *before* the tail call, so a function in tail position never appears in a backtrace. A failure deep in a tail-recursive chain reports an apparently shallow stack, and that is not a bug.
- The frame is pushed *after* the arguments are evaluated, so an error raised while computing an argument is attributed to the caller, not the callee.
- On the error path the `?` short-circuits and the frame is deliberately left standing - that is what a backtrace reads. A caller that swallows an error instead of propagating it must restore the stack itself with `truncate_call_frames`.

8 Errors and non-local exit

`eval` returns `Result<LispExp<T>, EvalError<T>>`. The error type is generic over the host context for one reason: so it can carry Lisp *values*. A thrown payload and the offending value in a type error are both `LispExp<T>`, which is only expressible if the error knows what `T` is.

What that bought Around forty sites used to write `got: format!("{:?}", value)` - rendering the offending value to a string and discarding it. That was tolerable while errors only reached Rust. Once `condition-case` puts them in front of Lisp code, a handler that receives `"5"` instead of `5` can do nothing with it.

8.1 The error type

`EvalError<T>` has 31 variants in four groups:

GROUP	VARIANTS
Runtime failures	<code>UnboundVariable</code> <code>UndefinedFunction</code> <code>UnvalidFunctionCall</code> <code>WrongNumberOfArguments</code> <code>WrongArgumentType</code>
Malformed special forms	<code>QuoteNotOneArgument</code> <code>IfNoConditionProvided</code> <code>LetUnvalidBindingAt</code> <code>CondInvalidClause</code> <code>DefunMisplacedParamMarker</code> and a dozen more
Non-local exit	<code>Throw { tag, value }</code> , <code>Signal { symbol, data }</code>
Host and escape hatch	<code>OutOfFuel</code> , <code>RuntimeMessage(String)</code>

`WrongArgumentType` holds `got: LispExp<T>` - the value itself, not a rendering of it. `Throw` and `Signal` carry their payloads inline, so an uncaught throw can still be inspected in full by the host rather than arriving as an opaque marker.

8.2 Two mechanisms, deliberately different

There are two ways out of a computation that is not returning normally, and the whole design rests on keeping them apart:

	THROW / CATCH	SIGNAL / CONDITION-CASE
What it means	a <i>control transfer</i> - go there, with this value	a <i>failure</i> - something went wrong
Selected by	tag equality	condition symbol matching
Caught by	the innermost catch naming that tag	the first matching handler
Passes through	<code>condition-case</code> untouched	<code>catch</code> untouched
Runs unwind-protect cleanups	yes	yes

So a `condition-case` never swallows a `throw` aimed past it, and a `catch` never swallows an error that belongs to a handler further out. Getting that backwards is the classic way to make a non-local exit mechanism unusable.

```
;; the throw passes straight through the condition-case
(catch 'tag
  (condition-case nil (throw 'tag 11) (error 'wrong))) => 11

;; the error passes straight through the catch
(catch 'tag (undefined-fn)) => propagates UndefinedFunction
```

8.3 catch and throw

`(catch TAG BODY...)` evaluates TAG, then each body form. If a `throw` unwinds through it, the tag is compared and either claimed or allowed past:

```
match eval(form, env.clone(), ctx) {
  Ok(value) => result = value,
  Err(EvalError::Throw { tag: thrown, value }) => {
    if thrown == tag {
      ctx.truncate_call_frames(depth_before);
      return Ok(EvalStep::Done(value)); // claimed
    }
    return Err(EvalError::Throw { tag: thrown, value }); // someone else's
  }
  Err(err) => return Err(err),
}
```

Tag comparison is structural equality. For symbols and numbers that is identical to Emacs's `eq`; it differs only for compound tags such as lists, which nothing uses.

```
(catch 'done (throw 'done 42) 99)           => 42
(catch 'outer (catch 'inner (throw 'outer 5)) 99) => 5 ; inner lets it pass
(catch 'tag (catch 'tag (throw 'tag 1) 2) 3)   => 3 ; innermost wins
(catch 'tag (funcall 'throw 'tag 8))           => 8 ; throw is a function
```

8.4 condition-case

`(condition-case VAR BODY-FORM HANDLER...)`, where each HANDLER is `(CONDITION BODY...)`. A condition is a symbol, or a list of symbols; `t` and `error` match everything. VAR, unless `nil`, is bound in the handler to `(CONDITION-SYMBOL . DATA)`, exactly as in Emacs.

Two helpers map a Rust error onto that pair. `error_symbol` gives the condition:

ERROR	CONDITION SYMBOL
UnboundVariable	unbound-variable
UndefinedFunction	undefined-function
UnvalidFunctionCall, UncorrectFunctionDefinition	invalid-function
WrongNumberOfArguments	wrong-number-of-arguments
WrongArgumentType	wrong-type-argument
OutOfFuel	out-of-fuel
RuntimeMessage	runtime-error
Signal { symbol, .. }	whatever Lisp passed to signal
every malformed-special-form variant	error

`error_data` gives the rest of the pair - the offending value for a type error, the unbound name for a lookup failure, the expected and actual arity for a call, and whatever `signal` was handed.

```
(condition-case nil (undefined-fn) (error 'handled))      => handled
(condition-case e  (undefined-fn) (error (car e)))        => undefined-function
(condition-case e  (undefined-fn) (error (car (cdr e))))  => undefined-fn

;; a list of conditions, and first match wins
(condition-case nil (undefined-fn)
  ((out-of-fuel undefined-function) 'matched))            => matched

;; the handler's value is the condition-case's value
(condition-case e (car 5) (wrong-type-argument (+ 1 (nth 1 (cdr e))))) => 6
```

A handler for a condition that does not match is skipped, and if none matches the error propagates unchanged - `condition-case` is not a catch-all unless you write one.

8.5 Raising from Lisp

`(signal SYMBOL DATA)` raises a condition of your own naming, catchable like any built-in one. `(throw TAG VALUE)` transfers control. Both are functions living in the standard library rather than special forms, which is why `funcall` and `apply` work on them.

```
(condition-case e (signal 'my-error '(1 2)) (my-error (cdr e))) => (1 2)
(condition-case nil (signal 'my-error nil) (error 'caught))    => caught
```

8.6 What survives an unwind

`unwind-protect` runs its cleanup forms on every path out of the body - normal return, error, or throw. Two details matter for diagnostics:

- A failed body leaves its call frames standing on purpose, because that is what a backtrace reads. `unwind-protect` records the depth after the body and truncates back to it once the cleanups have run, so the cleanup's own frames do not pile up on top of the failure's.
- `catch` and `condition-case` both truncate to the depth they recorded *before* the protected form. Once a failure has been handled, its frames are no longer interesting, and leaving them would pollute the next unrelated backtrace.

```
(setq trace nil)
(catch 'tag
  (unwind-protect (throw 'tag 'thrown)
    (setq trace 'cleaned)))
trace => cleaned
```

9 The host contract

`LispContext` is the entire interface between the interpreter and its host. Every method except the first two has a no-op default, so a host opts in to only what it needs.

METHOD	REQUIRED	CALLED WHEN
<code>consume_fuel(&self, u32) -> Result<(), EvalError<Self>></code>	yes	first statement of every <code>eval_step</code>
<code>log_diagnostic(&self, &str)</code>	yes	non-fatal notices, including a failure in a spawned thread
<code>begin_thread_evaluation(&self)</code>	no	first thing on a thread created by <code>spawn</code>
<code>push_call_frame(&self, &str)</code>	no	entering a named call, after its arguments are evaluated
<code>pop_call_frame(&self)</code>	no	leaving it successfully, and before a tail call
<code>call_frame_depth(&self) -> usize</code>	no	before something that might fail recoverably
<code>truncate_call_frames(&self, usize)</code>	no	after recovering, to undo the frames left behind

The trait also requires `Clone + PartialEq + Debug + Send + Sync + 'static`. `Send + Sync` and `Clone` are what let `spawn` hand a copy of the context to a new thread; in the editor every field is an `Arc`, so cloning shares rather than duplicates.

A host that wants none of it is four lines:

```
#[derive(Clone, Debug, PartialEq)]
struct Host;

impl LispContext for Host {
    fn consume_fuel(&self, _amount: u32) -> Result<(), EvalError<Self>> { Ok(()) }
    fn log_diagnostic(&self, msg: &str) { eprintln!("{}", msg); }
}
```

`EvalError<Self>` needs no extra bound: `LispContext` already requires `Clone`, which implies `Sized`.

This is not hypothetical The interpreter's own test suite uses Rust's unit type `()` as its context, with exactly those two methods. If a test can run the language with no host at all, nothing editor-shaped has leaked into the interpreter.

10 Metering

`consume_fuel` is called once per reduction step. What that budget is, and when it refills, is host *policy*; `FuelMeter` is the *mechanism*, and it is deliberately host-agnostic.

```
pub const DEFAULT_FUEL: u32 = 10_000_000;

pub struct FuelMeter { budget: AtomicU32 }      // shared: the setting

pub struct Exhausted;                          // what `consume` reports

thread_local! {
    static FUEL: Cell<u32> = const { Cell::new(DEFAULT_FUEL) }; // private: the counter
    static DEPTH: Cell<u32> = const { Cell::new(0) };
}
```

The split is the point. The configured budget is shared and read once per scope; the remaining count is thread-local and touched on every single step. So the hot counter needs no synchronisation at all - a plain `Cell` load and store, rather than an atomic read-modify-write on a cache line other cores may be contending for. The `const` initialiser avoids the hidden "has this thread initialised it yet?" branch that a lazy one would add to every access.

CALL	EFFECT
<code>meter.begin()</code>	if <code>DEPTH == 0</code> , refill <code>FUEL</code> from budget; <code>DEPTH += 1</code> ; returns a <code>FuelScope</code> guard
<code>meter.consume(n)</code>	<code>FUEL.checked_sub(n)</code> , or <code>Err(Exhausted)</code>
<code>drop(scope)</code>	<code>DEPTH -= 1</code>
<code>meter.arm_thread()</code>	set this thread's <code>FUEL</code> from budget, without opening a scope
<code>meter.set_budget(n)</code>	change the setting, and top this thread's <code>FUEL</code> up to it

Refilling only at depth zero is what stops nested evaluation from laundering itself a fresh budget partway through an existing scope. `FuelScope` is an `RAII` guard rather than a paired begin/end precisely so an early return or a `?` cannot leave the depth stuck above zero, which would permanently prevent any further refill.

`consume` reports a bare `Exhausted` rather than an `EvalError`. That is deliberate: the meter is host-agnostic machinery with no business naming the interpreter's error type, which is generic over the context. Naming the failure is the host's job - `self.fuel.consume(amount).map_err(|_| EvalError::OutOfFuel)`.

Starting at `DEFAULT_FUEL` rather than zero means a thread that never opens a scope is still *bounded* - it fails safe, instead of either dying on its first step or running unmetered.

Why this is not just a recursion limit The trampoline means tail recursion does not grow the Rust stack, so a runaway loop would otherwise spin forever with no native limit to hit. Fuel is the only thing that stops (while t
1) from hanging the host.

11 Concurrency

11.1 spawn

```
let mut thread_ctx = ctx.clone();           // every field is an Arc in a real host
std::thread::spawn(move || {
    thread_ctx.begin_thread_evaluation(); // arm this thread's fuel counter
    let frame = Env::new_child(&lambda.env);
    for exp in &lambda.body {
        if let Err(err) = eval(exp, frame.clone(), &thread_ctx) {
            thread_ctx.log_diagnostic(&format!("[LISP thread] {err:?}"));
            break;
        }
    }
});
Done(nil)                                   // returns immediately, never joins
```

Values cross the thread boundary because every `LispExp` payload is an `Arc`, and `Env` guards its three maps with `RwLock`. `spawn` is fire-and-forget: it yields `nil` at once and there is no join, so the only way to observe the thread is through an effect - an `Atom` it writes, or a diagnostic it logs.

`begin_thread_evaluation` is what gives the new thread the host's configured budget. Without it the thread would silently run on the compile-time `DEFAULT_FUEL`, since a fresh thread starts at the `const` initialiser rather than inheriting its parent's remaining count.

```
(setq shared (make-atom 0))
(spawn (lambda () (reset shared 42)))
(deref shared) => 0 immediately, 42 once the thread has run
```

11.2 Fibers

A fiber is cooperative and stays on the same thread. `(fiber BODY...)` captures the body plus a fresh child environment as a suspended value:

```
pub struct FiberState<T: LispContext> {
    pub body: Vec<LispExp<T>>,
    pub env: Arc<Env<T>>,
    pub is_done: bool,
}
```

Resuming runs the next body form and hands back its value; resetting rewinds it. There is no thread and no preemption - the interpreter simply stops between forms. `(fiber)` with no body yields an already-finished fiber.

11.3 Atoms

`Atom` is `Arc<RwLock<LispExp<T>>>` and is the only mutable container in the language. It is also, consequently, the only way to build a value that contains itself - which is why the printer refuses to look inside one.

12 Embedding it

Everything above, end to end. The one line that reaches outside this document's scope is `setup_base_env`, which installs the standard library - without it there is no `+` to call.

```
use rsedit_core::lisp::{Env, EvalError, LispContext, LispExp, Parser, eval, setup_base_env};

#[derive(Clone, Debug, PartialEq)]
struct Host;

impl LispContext for Host {
    fn consume_fuel(&self, _amount: u32) -> Result<(), EvalError<Self>> { Ok(()) }
    fn log_diagnostic(&self, msg: &str) { eprintln!("{msg}"); }
}

fn main() {
    let env = Env::new_root();
    setup_base_env(env.clone()); // the standard library - out of scope here

    let source = "(progn (defun sq (x) (* x x)) (sq 7))";
    let mut parser = Parser::new(source);
    let ast = parser.next().expect("parse error");

    match eval(&ast, env, &Host) {
        Ok(value) => println!("{:?}", value), // => 49
        Err(err) => eprintln!("error: {err:?}"),
    }
}
```

To meter it, hold a `FuelMeter`, delegate `consume_fuel` to it, and open one scope per top-level evaluation:

```

#[derive(Clone, Debug)]
struct Host { fuel: Arc<FuelMeter> }

impl LispContext for Host {
    fn consume_fuel(&self, amount: u32) -> Result<(), EvalError<Self>> {
        self.fuel.consume(amount).map_err(|_| EvalError::OutOfFuel)
    }
    fn log_diagnostic(&self, msg: &str) { eprintln!("{msg}"); }
    fn begin_thread_evaluation(&self) { self.fuel.arm_thread(); }
}

// at each top-level entry point:
let _scope = host.fuel.begin();      // refills; the guard closes the scope on drop
let value = eval(&ast, env, &host)?;

```

Parsing several top-level forms from one string means calling `parser.next()` repeatedly; wrapping the source in `(progn ...)`, as the test suites do, is the shortcut.

13 Where the interpreter stands

Commit `e6de186`. This section exists so the difference between what is in the tree and what has merely been written is explicit.

13.1 In the tree

LANDED IN	WHAT
b5f0186	test suite refactor; machine-independent performance tests
2ec4575	<code>ConsCell</code> , <code>ConsIter</code> , <code>cons/proper_list/improper_list/iter/split_list</code> , <code>form_to_data/data_to_form</code> , reader and backquote wiring
abf3ad9	the list primitives ported to cons cells; the hand-written Debug printer; <code>DottedList</code> removed
b988601 e9ec281	the <code>List</code> variant renamed to <code>Form</code>
d4a5ac2	the flow diagram
6be3b73	five tests proving a spawned thread gets its own fuel budget, verified by mutation rather than only by passing
e6de186	<code>EvalError<T></code> generic over the context; <code>catch/throw/signal/condition-case</code> ; the <code>unwind-protect</code> frame fix; <code>FuelMeter</code> decoupled from the Lisp error type

Two results are worth stating outright. `cons` and `cdr` are $O(1)$ field clones rather than operations that rebuilt the whole spine, so accumulating a list is linear rather than quadratic - the performance test that used to assert the quadratic behaviour now asserts the linear one and measures about 2x when the length doubles. And Lisp code can now recover from failure, which it could not do at all before `e6de186`.

13.2 Written and verified, not yet applied

PATCH	WHAT IT ADDS	TESTS
<code>fuel-checkpoint-6</code>	the <code>set-command-fuel</code> primitive, with a floor that stops a zero budget from locking the editor out - the last step of the fuel work	242

This is the only outstanding patch. It predates `e6de186`, so it will need its signatures re-checked against the now-generic `EvalError` before it applies cleanly.

13.3 Agreed next, not yet written

- Symbol property lists* - `put / get / symbol-plist`. Nothing exists today, and it is the substrate for (interactive) specs, mode metadata and faces.
- Macro tooling* - `gensym`, `macroexpand`, `macroexpand-1`. A macro expanding to a `let` currently captures caller variables silently, and there is no way to inspect an expansion.

- *A hand-written `PartialEq`* - the derived one recurses once per cons cell, so `(equal a b)` overflows the Rust stack on a long list. Same class of problem as the printer, same fix.
- *Then the standard library* - sequence and string functions, the regex primitives (the crate is already a dependency but nothing is exposed to Lisp), and hash-table access, since the `Map` variant is currently unreachable from Lisp code.

Dynamic binding: decided against Scope stays lexical, as it already is. Modern Emacs keeps dynamic binding for `defvar`-declared specials, but now that `unwind-protect` and `catch / throw` exist, the same effect is a `dlet` macro written in Lisp - save the global, run the body under `unwind-protect`, restore. No special-variable flag, no binding stack, no change to `let`. It needs `gensym` for hygiene, which is why that comes first.